

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TECHNICAL PRESENTATION

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Technical Presentation
Weightage:	10% of CIA	Total Marks:	100 (1 Question × 100 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the given question through a technical presentation. Total Marks: 100.
- Clearly define the problem and explain the concept of algorithm growth functions.
- Present comparative analysis using appropriate representations such as graphs, tables, or charts.
- Use suitable tools (PowerPoint, visualization tools, or software) to support your explanation.
- Ensure technical accuracy, logical flow, and proper interpretation of results.
- Maintain clarity in communication, proper slide organization, and professional delivery.
- Time management, confidence, and audience engagement will be considered during evaluation.

Question Type	Focus Area	Questions	Marks
Technical Presentation	Analyze and compare growth functions using mathematical techniques and asymptotic concepts and with graphical and visual interpretation	Q1	100
GRAND TOTAL:		100 Marks	

SECTION A – Technical Presentation

CO1_AT3 – Technical Presentation Answers

Course: Design and Analysis of Algorithms (CSA06) | CO1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.

Q1. Asymptotic Analysis of Sorting Algorithms — Merge Sort vs Quick Sort

Problem Definition

When sorting large datasets, the choice of algorithm affects running time and memory. Merge Sort and Quick Sort are both $O(n \log n)$ “divide and conquer” sorters on average, but they behave very differently in the worst case and in practice. The problem is to determine which algorithm is more efficient under different data conditions.

Recurrences

- **Merge Sort:** $T(n) = 2T(n/2) + O(n) \rightarrow$ always $\Theta(n \log n)$ (best, average, worst)
- **Quick Sort:** depends on pivot balance
- Best/Average case: $T(n) = 2T(n/2) + O(n) \rightarrow \Theta(n \log n)$
- Worst case (already sorted array, bad pivot): $T(n) = T(n-1) + O(n) \rightarrow \Theta(n^2)$

Case-wise Comparison Table

Case	Merge Sort	Quick Sort
Best	$\Theta(n \log n)$	$\Theta(n \log n)$
Average	$\Theta(n \log n)$	$\Theta(n \log n)$
Worst	$\Theta(n \log n)$	$\Theta(n^2)$
Space	$O(n)$ auxiliary	$O(\log n)$ (in-place, recursion stack)
Stability	Stable	Not stable

Growth Visualization (relative operation count for $n \log n$ vs n^2)

n	$n \log n$	n^2
100	~664	10,000
1,000	~9,966	1,000,000
10,000	~132,877	100,000,000

As n grows, n^2 pulls away dramatically from $n \log n$ — this is why Quick Sort's worst case becomes dangerous for large, adversarial, or nearly-sorted inputs.

Justification / Conclusion

- **Quick Sort** is usually faster in practice because of low constant factors and in-place partitioning (cache-friendly), so it wins for average-case, memory-constrained situations, and randomized pivot selection.
- **Merge Sort** guarantees $\Theta(n \log n)$ in every case and is stable, so it is preferred when worst-case guarantees matter (e.g., real-time systems) or when stability is required (e.g., sorting records by multiple keys), and it parallelizes well (external sorting on large files).
- **Conclusion:** For large datasets with unpredictable ordering and strict performance guarantees, Merge Sort is safer; for average workloads where memory is limited, Quick Sort (with randomized pivot) is more efficient.

Q2. Master Theorem Application — $T(n) = 2T(n/2) + n$

Problem Definition

This recurrence models algorithms that split a problem into 2 subproblems of half size and do $O(n)$ extra work to combine results (e.g., Merge Sort).

Master Theorem Setup

General form: $T(n) = aT(n/b) + f(n)$

Here: $a = 2$, $b = 2$, $f(n) = n$

Compute the watershed function: $n^{(\log_b a)} = n^{(\log_2 2)} = n^1 = n$

Compare $f(n) = n$ with $n^{(\log_b a)} = n$:

$f(n) = \Theta(n^{(\log_b a)})$ → this is **Case 2** of the Master Theorem.

Case 2 Statement

If $f(n) = \Theta(n^{(\log_b a)})$, then: $T(n) = \Theta(n^{(\log_b a)} \cdot \log n)$

Applying It

$T(n) = \Theta(n \cdot \log n) = \Theta(n \log n)$

Recursion Tree Cross-Check

- Level 0: cost n
- Level 1: two subproblems of size $n/2$, cost $n/2$ each → total n
- Level 2: four subproblems of size $n/4$ → total n
- ... every level costs n
- Number of levels = $\log_2 n$ (since size halves each time until reaching 1)
- Total cost = $n \times \log_2 n = \Theta(n \log n)$ matches Master Theorem result

Interpretation

Because the work done at the “combine” step ($f(n) = n$) exactly matches the rate at which subproblems multiply, the total cost is n multiplied by the number of levels, giving the characteristic $n \log n$ complexity seen in Merge Sort.

Q3. Substitution Method Analysis — $T(n) = T(n-1) + n$

Problem Definition

This recurrence models an algorithm that reduces the problem size by 1 at each step and performs $O(n)$ work at that step (e.g., selection-sort-style linear scan repeated n times).

Step-by-step Expansion

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + (n-1)$$

$$T(n-2) = T(n-3) + (n-2)$$

...

Substituting repeatedly:

$$T(n) = T(n-2) + (n-1) + n$$

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

...

$$T(n) = T(n-k) + [(n-k+1) + (n-k+2) + \dots + n]$$

Base Case

Stop when $n - k = 1$, i.e., $k = n - 1$, giving $T(1) = c$ (constant).

$$T(n) = T(1) + (2 + 3 + \dots + n)$$

$$T(n) = c + \sum_{i=2}^n i$$

$$T(n) = c + [n(n+1)/2 - 1]$$

Simplification

$T(n) = O(n^2)$ since the sum of the first n natural numbers is $n(n+1)/2$, which is quadratic in n .

Final result: $T(n) = \Theta(n^2)$

Verification (Substitution / Induction Check)

Guess $T(n) = O(n^2)$. Assume $T(k) \leq ck^2$ for all $k < n$.

$$T(n) = T(n-1) + n \leq c(n-1)^2 + n = c \cdot n^2 - 2cn + c + n$$

For this to be $\leq cn^2$, we need: $-2cn + c + n \leq 0$, which holds for sufficiently large c and n . Confirms $T(n) = O(n^2)$.

Interpretation

This recurrence, growing quadratically, corresponds to a decrementing linear-scan pattern — inefficient for large n , since each of the n steps does linear work, giving $n \times n = n^2$ total operations.

Q4. Comparison of Growth Rates — $\log n$, n , $n \log n$, n^2 , 2^n

Problem Definition

Selecting an efficient algorithm requires understanding how different complexity classes scale as input size n grows. This determines which algorithms remain usable for large n .

Comparative Growth Table (approximate operation counts)

n	$\log n$	n	$n \log n$	n^2	2^n
10	3.3	10	33	100	1,024
20	4.3	20	86	400	1,048,576
50	5.6	50	282	2,500	$\sim 10^{15}$
100	6.6	100	664	10,000	$\sim 10^{30}$

Ranking (slowest to fastest growth)

$$\log n < n < n \log n < n^2 < 2^n$$

Explanation

- **$\log n$** : grows extremely slowly — doubling n barely increases the value (e.g., binary search).
- **n** : grows proportionally with input (e.g., linear scan).
- **$n \log n$** : slightly worse than linear, typical of efficient sorting/divide-and-conquer algorithms.
- **n^2** : grows quadratically — becomes impractical for large n (e.g., nested loops, bubble sort).
- **2^n** : grows explosively — even moderate n (e.g., $n = 50$) becomes computationally infeasible (e.g., brute-force subset enumeration).

Justification / Conclusion

For large-scale systems, algorithms with **$\log n$ or $n \log n$** complexity are strongly preferred. **n^2** algorithms are acceptable only for small/medium inputs. **2^n (exponential)** algorithms are impractical beyond small n and typically require heuristics, approximation, or dynamic programming to become tractable.

Q5. Recursion Tree Method — $T(n) = 3T(n/2) + n$

Problem Definition

This recurrence describes an algorithm that splits a problem into 3 subproblems, each of half size, plus $O(n)$ work to combine — more subproblems than Merge Sort, so this grows faster than $n \log n$.

Building the Recursion Tree

- **Level 0:** 1 problem of size $n \rightarrow \text{cost} = n$
- **Level 1:** 3 problems of size $n/2 \rightarrow \text{cost} = 3 \times (n/2) = (3/2)n$
- **Level 2:** 9 problems of size $n/4 \rightarrow \text{cost} = 9 \times (n/4) = (3/2)^2 n$
- **Level i :** 3^i problems of size $n/2^i \rightarrow \text{cost} = 3^i \times (n/2^i) = (3/2)^i \times n$

Number of Levels

The tree ends when subproblem size = 1: $n/2^i = 1 \rightarrow i = \log_2 n$

Total Cost (sum across all levels)

$$T(n) = \sum_{i=0}^{\log_2 n} (3/2)^i \times n$$

This is a geometric series with ratio $r = 3/2 > 1$, so the **last level dominates**.

$$\text{Total} = n \times [(3/2)^{(\log_2 n + 1)} - 1] / (3/2 - 1)$$

$$\text{Since } (3/2)^{(\log_2 n)} = n^{(\log_2 1.5)} = n^{(\log_2 3 - 1)}:$$

$$T(n) = \Theta(n^{(\log_2 3)}) \approx \Theta(n^{1.585})$$

Cross-Check with Master Theorem

$$a = 3, b = 2, f(n) = n$$

$$n^{(\log_2 b/a)} = n^{(\log_2 3)} \approx n^{1.585}$$

Since $f(n) = n = O(n^{(1.585 - \epsilon)})$ for some $\epsilon > 0$, this is **Case 1**:

$$T(n) = \Theta(n^{(\log_2 b/a)}) = \Theta(n^{1.585}) \quad \text{matches recursion tree result}$$

Interpretation

Because each level multiplies the number of subproblems by 3 while size only halves, the tree grows “wide” faster than it grows “short” — most of the total work is concentrated in the leaves (the last level), giving a complexity between n and n^2 (specifically $n^{1.585}$), worse than the $\Theta(n \log n)$ of Merge Sort's $2T(n/2)+n$ recurrence.